

labassignment7

March 18, 2025

1 Lab Assignment 7: Database Queries

1.1 DS 6001: Practice and Application of Data Science

1.1.1 Clay Harris jbm2rt

1.1.2 Instructions

Please answer the following questions as completely as possible using text, code, and the results of code as needed. Format your answers in a Jupyter notebook. To receive full credit, make sure you address every part of the problem, and make sure your document is formatted in a clean and professional way.

1.1.3 Problem 0

Import the following libraries, load the `.env` file where you store your passwords (see the notebook for module 4 for details), and turn off the error tracebacks to make errors easier to read:

```
[61]: import numpy as np
import pandas as pd
import sys
import os
import requests
import psycpg2
import pymongo
from pymongo import MongoClient
import json
import psycpg
from bson.json_util import dumps, loads
from sqlalchemy import create_engine
from sqlalchemy import text
import dotenv

# change to the directory where your .env file is
os.chdir("/Users/queclay/Documents/MSDS/DS6001/M06/ds6001databases/")

dotenv.load_dotenv() # register the .env file where passwords are stored
sys.tracebacklimit = 0 # turn off the error tracebacks
```

1.1.4 Problem 1

For this problem, we will be building a PostgreSQL database that contains the collected works of Shakespeare.

The data were collected by [Catherine Devlin](#) from the repository at <https://opensourceshakespeare.org/>. The database will have four tables, one representing works by Shakespeare, one for characters that appear in Shakespeare's plays, one for chapters (this is, scenes within acts), and one for paragraphs (that is, lines of dialogue). The data to populate these four tables are here:

```
[3]: works = pd.read_csv("https://github.com/jkropko/DS-6001/raw/master/localdata/
↳Works.csv")
characters = pd.read_csv("https://github.com/jkropko/DS-6001/raw/master/
↳localdata/Characters.csv")
chapters = pd.read_csv("https://github.com/jkropko/DS-6001/raw/master/localdata/
↳Chapters.csv")
paragraphs = pd.read_csv("https://github.com/jkropko/DS-6001/raw/master/
↳localdata/Paragraphs.csv")
```

In PostgreSQL, it is best practice to convert all column names to lower-case, as case sensitive column names will require [extraneous double-quotes](#) in any query. We first convert the column names in all four dataframe to lowercase:

```
[4]: works.columns = works.columns.str.lower()
characters.columns = characters.columns.str.lower()
chapters.columns = chapters.columns.str.lower()
paragraphs.columns = paragraphs.columns.str.lower()
```

You will build a database and populate it with these data. The ER diagram for the database is:

There's no codebook, unfortunately, but the values in the columns are mostly self-explanatory:

```
[15]: works.head()
```

```
[15]:
```

	workid		title	longtitle \
0	12night		Twelfth Night	Twelfth Night, Or What You Will
1	allswell	All's Well That Ends Well		All's Well That Ends Well
2	antonycleo	Antony and Cleopatra		Antony and Cleopatra
3	asyoulikeit	As You Like It		As You Like It
4	comedyerrors	Comedy of Errors		The Comedy of Errors

	date	genretype	notes	source	totalwords	totalparagraphs
0	1599	c	NaN	Moby	19837	1031
1	1602	c	NaN	Moby	22997	1025
2	1606	t	NaN	Moby	24905	1344
3	1599	c	NaN	Gutenberg	21690	872
4	1589	c	NaN	Moby	14692	661

```
[16]: characters.head()
```

```
[16]:
```

	charid	charname	abbrev	works \
0	1apparition-mac	First Apparition	First Apparition	macbeth
1	1citizen	First Citizen	First Citizen	romeojuliet
2	1conspirator	First Conspirator	First Conspirator	coriolanus
3	1gentleman-oth	First Gentleman	First Gentleman	othello
4	1goth	First Goth	First Goth	titus

	description	speechcount
0	NaN	1.0
1	NaN	3.0
2	NaN	3.0
3	NaN	1.0
4	NaN	4.0

```
[17]: chapters.head()
```

```
[17]:
```

	workid	chapterid	section	chapter	description
0	12night	18704.0	1.0	1.0	DUKE ORSINO's palace.
1	12night	18705.0	1.0	2.0	The sea-coast.
2	12night	18706.0	1.0	3.0	OLIVIA'S house.
3	12night	18707.0	1.0	4.0	DUKE ORSINO's palace.
4	12night	18708.0	1.0	5.0	OLIVIA'S house.

```
[18]: paragraphs.head()
```

```
[18]:
```

	workid	paragraphid	paragraphnum	charid \
0	12night	630863	3	xxx
1	12night	630864	4	ORSINO
2	12night	630865	19	CURIO
3	12night	630866	20	ORSINO
4	12night	630867	21	CURIO

```

                                plaintext \
0 [Enter DUKE ORSINO, CURIO, and other Lords; Mu...
1 If music be the food of love, play on;\n[p]Giv...
2 Will you go hunt, my lord?\n
3 What, Curio?\n
4 The hart.\n

```

```

                                phoneticictext \
0 ENTR TK ORSN KR ANT OOR LRTS MSXNS ATNTNK
1 IF MSK B O FT OF LF PL ON JF M EKSSS OF IT OT ...
2 WL Y K HNT M LRT
3 HT KR
4 O HRT

```

```
stemtext paragraphtype section \
```

0	enter duke orsino curio and other lord musicia...	b	1.0
1	if music be the food of love plai on give me e...	b	1.0
2	will you go hunt my lord	b	1.0
3	what curio	b	1.0
4	the hart	b	1.0

	chapter	charcount	wordcount
0	1.0	65.0	9.0
1	1.0	646.0	114.0
2	1.0	27.0	6.0
3	1.0	13.0	2.0
4	1.0	10.0	2.0

Part a Connect to your local PostgreSQL server (take steps to hide your password!), create a new database for the Shakespeare data, use `create_engine()` from `sqlalchemy` to connect to the database, and create the works, characters, chapters, and paragraphs tables populated with the data from the four dataframes shown above. [2 points]

```
[34]: POSTGRES_USER = "postgres"
POSTGRES_PASSWORD = os.getenv("POSTGRES_PASSWORD")
POSTGRES_HOST = "localhost"
POSTGRES_PORT = "5432"
POSTGRES_DBNAME = "shakespeare"

with psycopg.connect(
    host=POSTGRES_HOST,
    user=POSTGRES_USER,
    password=POSTGRES_PASSWORD,
    dbname="postgres",
    port=POSTGRES_PORT,
    autocommit=True
) as conn:
    conn.execute(f"""
        SELECT pg_terminate_backend(pid)
        FROM pg_stat_activity
        WHERE datname = '{POSTGRES_DBNAME}'
        AND pid <> pg_backend_pid();
    """)

    conn.execute(f"DROP DATABASE IF EXISTS {POSTGRES_DBNAME};")
    conn.execute(f"CREATE DATABASE {POSTGRES_DBNAME};")

    print(f"Database '{POSTGRES_DBNAME}' dropped and recreated successfully.")

with psycopg.connect(
    host=POSTGRES_HOST,
    user=POSTGRES_USER,
```

```

password=POSTGRES_PASSWORD,
dbname=POSTGRES_DBNAME,
port=POSTGRES_PORT,
autocommit=True
) as conn:

conn.execute("""
CREATE TABLE works (
    workid VARCHAR PRIMARY KEY,
    title VARCHAR,
    longtitle VARCHAR,
    date INT,
    genretype VARCHAR(1),
    notes TEXT,
    source TEXT,
    totalwords INT,
    totalparagraphs INT
);

CREATE TABLE characters (
    charid VARCHAR PRIMARY KEY,
    charname VARCHAR,
    abbrev VARCHAR,
    description TEXT,
    speechcount INT
);

CREATE TABLE character_work (
    charid VARCHAR REFERENCES characters(charid),
    workid VARCHAR REFERENCES works(workid),
    PRIMARY KEY (charid, workid)
);

CREATE TABLE chapters (
    chapterid VARCHAR PRIMARY KEY,
    workid VARCHAR REFERENCES works(workid),
    section VARCHAR,
    chapter VARCHAR,
    description TEXT
);

CREATE TABLE paragraphs (
    paragraphid VARCHAR PRIMARY KEY,
    charid VARCHAR REFERENCES characters(charid),
    workid VARCHAR REFERENCES works(workid),
    section VARCHAR,
    chapter VARCHAR,

```

```

        paragraphnum INT,
        plaintext TEXT,
        phonetictext TEXT,
        stemtext TEXT,
        paragraptype VARCHAR,
        charcount INT,
        wordcount INT
    );
    """

    print("Tables created with correct data types and foreign keys.")

engine = create_engine(
    f"postgresql+psycpg://{{POSTGRES_USER}}:{{POSTGRES_PASSWORD}}@{{POSTGRES_HOST}}:
    ↪{{POSTGRES_PORT}}/{{POSTGRES_DBNAME}}"
)

characters.rename(columns={"works": "workid"}, inplace=True)

char_work_records = []

for _, row in characters.iterrows():
    workids = row['workid']

    if pd.isna(workids):
        continue

    for workid in map(str.strip, workids.split(", ")):
        char_work_records.append({
            'charid': row['charid'],
            'workid': workid
        })

character_work_df = pd.DataFrame(char_work_records)

characters_clean = characters.drop(columns=['workid'])

chapters_clean = chapters.dropna(subset=['chapterid']).copy()
chapters_clean['chapterid'] = chapters_clean['chapterid'].astype(str)

paragraphs_clean = (
    paragraphs[paragraphs['paragraphnum'].astype(str).str.isnumeric()]
    .dropna(subset=['section', 'chapter', 'charcount', 'wordcount'])
    .copy()
)

int_columns = ['paragraphnum', 'section', 'chapter', 'charcount', 'wordcount']

```

```

for col in int_columns:
    paragraphs_clean[col] = paragraphs_clean[col].astype(int)

works.to_sql("works", engine, if_exists="append", index=False)
characters_clean.to_sql("characters", engine, if_exists="append", index=False)
character_work_df.to_sql("character_work", engine, if_exists="append",
    ↪index=False)
chapters_clean.to_sql("chapters", engine, if_exists="append", index=False)
paragraphs_clean.to_sql("paragraphs", engine, if_exists="append", index=False)

print("Data inserted successfully into tables.")

```

Database 'shakespeare' dropped and recreated successfully.
 Tables created with correct data types and foreign keys.
 Data inserted successfully into tables.

Part b Write a query to display title, date, and totalwords from the works table. Rename date to year, and sort the output by totalwords in descending order. Also create a new column called era which is equal to “early” for works created before 1600, “middle” for works created between 1600 and 1607, and “late” for works created after 1607. Finally, display only the 7th through 11th rows of the output data. [1 point]

```

[ ]: query = """
SELECT
    title,
    date AS year,
    totalwords,
    CASE
        WHEN date < 1600 THEN 'early'
        WHEN date BETWEEN 1600 AND 1607 THEN 'middle'
        WHEN date > 1607 THEN 'late'
    END AS era
FROM works
ORDER BY totalwords DESC
OFFSET 6
LIMIT 5;
"""

result = pd.read_sql_query(query, engine)
result

```

```

[ ]:

```

	title	year	totalwords	era
0	King Lear	1605	26119	middle
1	Troilus and Cressida	1601	26089	middle
2	Henry IV, Part II	1597	25692	early
3	Henry VI, Part II	1590	25411	early
4	The Winter's Tale	1610	24914	late

Part c The `genretype` column in the “works” table designates five types of Shakespearean work:

- `t` is a tragedy, such as *Romeo and Juliet* and *Hamlet*
- `c` is a comedy, such as *A Midsummer Night’s Dream* and *As You Like It*
- `h` is a history, such as *Henry V* and *Richard III*
- `s` refers to Shakespeare’s sonnets
- `p` is a narrative (non-sonnet) poem, such as *Venus and Adonis* and *Passionate Pilgrim*

Write a query that generates a table that reports the average number of words in Shakespeare’s works by genre type. Display the genre type and the average wordcount within genre, use appropriate aliases, and sort by the average in descending order. [1 point]

```
[37]: query = """
SELECT
    CASE
        WHEN genretype = 't' THEN 'Tragedy'
        WHEN genretype = 'c' THEN 'Comedy'
        WHEN genretype = 'h' THEN 'History'
        WHEN genretype = 's' THEN 'Sonnet'
        WHEN genretype = 'p' THEN 'Poem'
    END AS genre,
    AVG(totalwords) AS average_wordcount
FROM works
GROUP BY genre
ORDER BY average_wordcount DESC;
"""

result = pd.read_sql_query(query, engine)
result
```

```
[37]:      genre  average_wordcount
0  History      24236.000000
1  Tragedy      23817.363636
2   Comedy      20212.071429
3   Sonnet      17515.000000
4    Poem       6181.800000
```

Part d Use a query to generate a table that contains the text of Hamlet’s (the character, not just the play) longest speech, and use the `print()` function to display this text. [1 point]

```
[42]: hamlet_longest_paragraph_query = """
SELECT
    paragraphid,
    workid,
    charid,
    plaintext,
    wordcount
FROM paragraphs
```



```

WHERE LOWER(charid) = 'hamlet'
ORDER BY wordcount DESC
LIMIT 1;
"""

hamlet_longest_paragraph_df = pd.read_sql_query(hamlet_longest_paragraph_query,
↪engine)

print(hamlet_longest_paragraph_df['plaintext'].iloc[0])

```

Ay, so, God b' wi' ye! [Exeunt Rosencrantz and Guildenstern]

[p]Now I am alone.

[p]O what a rogue and peasant slave am I!

[p]Is it not monstrous that this player here,
[p]But in a fiction, in a dream of passion,
[p]Could force his soul so to his own conceit
[p]That, from her working, all his visage wann'd,
[p]Tears in his eyes, distraction in's aspect,
[p]A broken voice, and his whole function suiting
[p]With forms to his conceit? And all for nothing!
[p]For Hecuba!

[p]What's Hecuba to him, or he to Hecuba,
[p]That he should weep for her? What would he do,
[p]Had he the motive and the cue for passion
[p]That I have? He would drown the stage with tears
[p]And cleave the general ear with horrid speech;
[p]Make mad the guilty and appal the free,
[p]Confound the ignorant, and amaze indeed
[p]The very faculties of eyes and ears.
[p]Yet I,
[p]A dull and muddy-mettled rascal, peak
[p]Like John-a-dreams, unpregnant of my cause,
[p]And can say nothing! No, not for a king,
[p]Upon whose property and most dear life
[p]A damn'd defeat was made. Am I a coward?
[p]Who calls me villain? breaks my pate across?
[p]Plucks off my beard and blows it in my face?
[p]Tweaks me by th' nose? gives me the lie i' th' throat
[p]As deep as to the lungs? Who does me this, ha?
[p]'Swounds, I should take it! for it cannot be
[p]But I am pigeon-liver'd and lack gall
[p]To make oppression bitter, or ere this
[p]I should have fatted all the region kites
[p]With this slave's offal. Bloody bawdy villain!
[p]Remorseless, treacherous, lecherous, kindless villain!
[p]O, vengeance!

[p]Why, what an ass am I! This is most brave,

[p]That I, the son of a dear father murther'd,
 [p]Prompted to my revenge by heaven and hell,
 [p]Must (like a whore) unpack my heart with words
 [p]And fall a-cursing like a very drab,
 [p]A scullion!
 [p]Fie upon't! foh! About, my brain! Hum, I have heard
 [p]That guilty creatures, sitting at a play,
 [p]Have by the very cunning of the scene
 [p]Been struck so to the soul that presently
 [p]They have proclaim'd their malefactions;
 [p]For murther, though it have no tongue, will speak
 [p]With most miraculous organ, I'll have these Players
 [p]Play something like the murther of my father
 [p]Before mine uncle. I'll observe his looks;
 [p>I'll tent him to the quick. If he but blench,
 [p>I know my course. The spirit that I have seen
 [p>May be a devil; and the devil hath power
 [p>T' assume a pleasing shape; yea, and perhaps
 [p>Out of my weakness and my melancholy,
 [p>As he is very potent with such spirits,
 [p>Abuses me to damn me. I'll have grounds
 [p>More relative than this. The play's the thing
 [p>Wherein I'll catch the conscience of the King. Exit.

1.1.5 Part e

Many scenes in Shakespeare's works take place in palaces or castles. Use a query to create a table that lists all of the chapters that take place in a palace. Include the work's title, the section (renamed to "act"), the chapter (renamed to "scene"), and the description of these chapters. The setting of each scene is listed in the `description` column of the "chapters" table. [Hint: be sure to account for case sensitivity] [2 points]

```
[ ]: palace_scenes_query = text("""
SELECT
    w.title,
    c.section AS act,
    c.chapter AS scene,
    c.description
FROM chapters c
JOIN works w
    ON c.workid = w.workid
WHERE
    LOWER(c.description) LIKE '%palace%'
ORDER BY w.title, act, scene;
""")

palace_scenes_df = pd.read_sql_query(palace_scenes_query, con=engine)
```

```
palace_scenes_df
```

```
[ ]:
      title act scene \
0    All's Well That Ends Well  1    1
1    All's Well That Ends Well  1    2
2    All's Well That Ends Well  1    3
3    All's Well That Ends Well  2    1
4    All's Well That Ends Well  2    2
..
120   Two Gentlemen of Verona  2    6
121   Two Gentlemen of Verona  3    1
122   Two Gentlemen of Verona  3    2
123   Two Gentlemen of Verona  4    2
124   Two Gentlemen of Verona  5    2

      description
0    Rousillon. The COUNT's palace.
1           Paris. The KING's palace.
2    Rousillon. The COUNT's palace.
3           Paris. The KING's palace.
4    Rousillon. The COUNT's palace.
..
120           The same. The DUKE'S palace.
121           Milan. The DUKE's palace.
122           The same. The DUKE's palace.
123 Milan. Outside the DUKE's palace, under SILVIA...
124           The same. The DUKE's palace.

[125 rows x 4 columns]
```

1.1.6 Part f

Create a table that lists characters, the plays that the characters appear in, the number of speeches the character gives, and the average length of the speeches that the character gives. Display the character description and the work title, not the ID values. Sort the table by average speech length, and restrict the table to only those characters that give at least 20 speeches. [Hint: you will need to use a subquery.] [2 points]

```
[56]: query = """
SELECT
    c.charname AS character_name,
    c.description AS character_description,
    STRING_AGG(DISTINCT w.title, '; ') AS play_titles,
    COUNT(p.paragraphid) AS total_speeches,
    ROUND(AVG(p.wordcount), 2) AS avg_speech_length
FROM
    characters c
```

```

JOIN
    character_work cw
    ON c.charid = cw.charid
JOIN
    works w
    ON cw.workid = w.workid
JOIN
    paragraphs p
    ON p.charid = c.charid AND p.workid = w.workid
WHERE
    w.genretype IN ('t', 'c', 'h')
    AND p.paragraphtype = 'b'
GROUP BY
    c.charname, c.description
HAVING
    COUNT(p.paragraphid) >= 20
ORDER BY
    avg_speech_length DESC;
"""

result_df = pd.read_sql_query(query, engine)
result_df

```

```

[56]:
      character_name      character_description \
0      King Richard II      king of England
1      Queen Katharine  wife to King Henry, afterwards divorced
2      Constance      mother to Arthur
3      Third Gentleman      None
4      Oberon      king of the fairies
..      ...      ...
387      Curtis      None
388      Lucius      servant to Brutus
389      Alice      a lady attending on Princess Katherine
390      All      None
391  (stage directions)      None

      play_titles  total_speeches \
0      Richard II      98
1      Henry VIII      50
2      King John      36
3      Henry VIII; Othello; The Winter's Tale      20
4      Midsummer Night's Dream      29
..      ...      ...
387      Taming of the Shrew      20
388      Julius Caesar      24
389      Henry V      22
390  All's Well That Ends Well; Antony and Cleopatr...      90

```

	avg_speech_length
0	61.77
1	59.36
2	59.22
3	57.10
4	55.66
..	...
387	8.55
388	8.54
389	7.45
390	5.89
391	4.06

[392 rows x 5 columns]

1.1.7 Part g

Which Shakespearean works do not contain any scenes in a palace or a castle? Use a query that displays the title, genre type, and publication date of works that do not contain any scenes that take place in a palace or castle. [Hint: use your work in part e as a starting point. You will need a subquery, and you will need to think carefully about the type of join that you need to perform.][2 points]

```
[ ]: works_no_palace_castle_query = text("""
SELECT
    w.title,
    CASE
        WHEN w.genretype = 't' THEN 'Tragedy'
        WHEN w.genretype = 'c' THEN 'Comedy'
        WHEN w.genretype = 'h' THEN 'History'
        WHEN w.genretype = 's' THEN 'Sonnet'
        WHEN w.genretype = 'p' THEN 'Poem'
        ELSE 'Unknown'
    END AS genre,
    w.date
FROM
    works w
WHERE
    w.workid NOT IN (
        SELECT DISTINCT c.workid
        FROM chapters c
        WHERE
            LOWER(c.description) LIKE '%palace%'
            OR LOWER(c.description) LIKE '%castle%'
    )
    )
```

```
ORDER BY
    w.title;
""")

works_no_palace_castle_df = pd.read_sql_query(works_no_palace_castle_query,
    engine)

works_no_palace_castle_df
```

```
[ ]:
      title      genre  date
0      Coriolanus  Tragedy  1607
1      Julius Caesar  Tragedy  1599
2      Lover's Complaint    Poem  1609
3      Love's Labour's Lost    Comedy  1594
4      Merchant of Venice    Comedy  1596
5      Merry Wives of Windsor    Comedy  1600
6      Much Ado about Nothing    Comedy  1598
7      Passionate Pilgrim    Poem  1598
8      Phoenix and the Turtle    Poem  1601
9      Rape of Lucrece    Poem  1594
10     Romeo and Juliet  Tragedy  1594
11     Sonnets    Sonnet  1609
12     Taming of the Shrew    Comedy  1593
13     Tempest    Comedy  1611
14     Timon of Athens  Tragedy  1607
15     Venus and Adonis    Poem  1593
```

1.1.8 Problem 2

The following file contains JSON formatted data of the official English-language translations of every constitution currently in effect in the world:

```
[8]: const = requests.get("https://github.com/jkropko/DS-6001/raw/master/localdata/
    const.json")
const_json = json.loads(const.text)
pd.DataFrame.from_records(const_json)
```

```
[8]:
      text      country \
0  'Afghanistan 2004  Preamble  \nIn the na...  Afghanistan
1  'Albania 1998 (rev. 2012)  Preamble  \nWe...  Albania
2  'Andorra 1993  Preamble  \nThe Andorran P...  Andorra
3  'Angola 2010  Preamble  \nWe, the people ...  Angola
4  'Antigua and Barbuda 1981  Preamble  \nWH...  Antigua and Barbuda
..
140  'Uzbekistan 1992 (rev. 2011)  Preamble  \n...  Uzbekistan
141  'Viet Nam 1992 (rev. 2013)  Preamble  \nI...  Viet Nam
142  'Yemen 1991 (rev. 2001)  PART ONE. THE FOUN...
```

```

143 'Zambia 1991 (rev. 2009)      Preamble    \nWE,...           Zambia
144 'Zimbabwe 2013              Preamble    \nWe the people...  Zimbabwe

```

```

        adopted revised reinstated democracy
0         2004      NaN          NaN  0.372201
1         1998    2012.0          NaN  0.535111
2         1993      NaN          NaN      NaN
3         2010      NaN          NaN  0.315043
4         1981      NaN          NaN      NaN
..        ...      ...          ...      ...
140        1992    2011.0          NaN  0.195932
141        1992    2013.0          NaN  0.251461
142        1991    2001.0          NaN  0.125708
143        1991    2009.0          NaN  0.405497
144        2013      NaN          NaN  0.315359

```

```
[145 rows x 6 columns]
```

The text of the constitutions are available from the [Wolfram Data Repository](#). I also included scores that represent the level of democratic quality in each country as of 2016. These scores are compiled by the [Varieties of Democracy \(V-Dem\)](#) project. Higher scores indicate greater levels of democratic openness and competition.

Part a Connect to your local MongoDB server and create a new collection for the constitution data. Use `.delete_many({})` to remove any existing data from this collection, and insert the data in `const_json` into this collection. [2 points]

```

[68]: MONGO_USER = os.getenv("MONGO_INITDB_ROOT_USERNAME")
MONGO_PASS = os.getenv("MONGO_INITDB_ROOT_PASSWORD")

MONGO_HOST = "localhost"
MONGO_PORT = 27017

mongo_uri = f"mongodb://{MONGO_USER}:{MONGO_PASS}@{MONGO_HOST}:{MONGO_PORT}/"

client = MongoClient(mongo_uri)

db = client['constitutions_db']
collection = db['constitutions']

collection.delete_many({})

collection.insert_many(const_json)

print("Constitution data inserted successfully.")

```

Constitution data inserted successfully.

Part b Use MongoDB queries and the `dumps()` and `loads()` functions from the `bson` package to produce dataframes with the following restrictions:

- The country, adoption year, and democracy features (and not `_id`, `text`, `revised`, or `reinstated`) for countries with constitutions that were written after 1990
- The country, adoption year, and democracy features (and not `_id`, `text`, `revised`, or `reinstated`) for countries with constitutions that were written after 1990 AND have a democracy score of less than 0.5
- The country, adoption year, and democracy features (and not `_id`, `text`, `revised`, or `reinstated`) for countries with constitutions that were written after 1990 OR have a democracy score of less than 0.5

[1 point]

```
[ ]: projection = {
    "_id": 0,
    "country": 1,
    "adopted": 1,
    "democracy": 1
}

query_1 = {"adopted": {"$gt": 1990}}
results_1 = collection.find(query_1, projection)
df_1 = pd.DataFrame.loads(dumps(results_1))

query_2 = {
    "$and": [
        {"adopted": {"$gt": 1990}},
        {"democracy": {"$lt": 0.5}}
    ]
}
results_2 = collection.find(query_2, projection)
df_2 = pd.DataFrame.loads(dumps(results_2))

query_3 = {
    "$or": [
        {"adopted": {"$gt": 1990}},
        {"democracy": {"$lt": 0.5}}
    ]
}
results_3 = collection.find(query_3, projection)
df_3 = pd.DataFrame.loads(dumps(results_3))

print("Adopted after 1990:")
display(df_1)

print("Adopted after 1990 AND democracy < 0.5:")
display(df_2)
```



```
print("Adopted after 1990 OR democracy < 0.5:")
display(df_3)
```

Adopted after 1990:

	country	adopted	democracy
0	Afghanistan	2004	0.372201
1	Albania	1998	0.535111
2	Andorra	1993	NaN
3	Angola	2010	0.315043
4	Armenia	1995	0.393278
..	...	...	...
66	Uzbekistan	1992	0.195932
67	Viet Nam	1992	0.251461
68	Yemen	1991	0.125708
69	Zambia	1991	0.405497
70	Zimbabwe	2013	0.315359

[71 rows x 3 columns]

Adopted after 1990 AND democracy < 0.5:

	country	adopted	democracy
0	Afghanistan	2004	0.372201
1	Angola	2010	0.315043
2	Armenia	1995	0.393278
3	Belarus	1994	0.289968
4	Bosnia and Herzegovina	1995	0.338267
5	Cambodia	1993	0.313738
6	Egypt	2014	0.218600
7	Equatorial Guinea	1991	0.217861
8	Eritrea	1997	0.075621
9	Ethiopia	1994	0.254865
10	Fiji	2013	0.473559
11	Gambia	1996	0.348132
12	Iraq	2005	0.455402
13	Kazakhstan	1995	0.262596
14	Lao People's Democratic Republic	1991	0.094434
15	Libya	2011	0.294716
16	Maldives	2008	0.386754
17	Montenegro	2007	0.455338
18	Myanmar	2008	0.405772
19	Oman	1996	0.191211
20	Russian Federation	1993	0.275516
21	Rwanda	2003	0.274476
22	Saudi Arabia	1992	0.024049
23	Serbia	2006	0.474443
24	Somalia	2012	0.177772

25	South Sudan	2011	0.183267
26	Sudan	2005	0.311799
27	Swaziland	2005	0.136008
28	Syrian Arab Republic	2012	0.148212
29	Turkmenistan	2008	0.154887
30	Uganda	1995	0.338308
31	Ukraine	1996	0.361911
32	Uzbekistan	1992	0.195932
33	Viet Nam	1992	0.251461
34	Yemen	1991	0.125708
35	Zambia	1991	0.405497
36	Zimbabwe	2013	0.315359

Adopted after 1990 OR democracy < 0.5:

	country	adopted	democracy
0	Afghanistan	2004	0.372201
1	Albania	1998	0.535111
2	Andorra	1993	NaN
3	Angola	2010	0.315043
4	Armenia	1995	0.393278
..	...	...	...
78	Uzbekistan	1992	0.195932
79	Viet Nam	1992	0.251461
80	Yemen	1991	0.125708
81	Zambia	1991	0.405497
82	Zimbabwe	2013	0.315359

[83 rows x 3 columns]

Part c According to the Varieties of Democracy project, [Hungary has become less democratic](#) over the last few years, and can no longer be considered a democracy. Update the record for Hungary to set the democracy score at 0.4. Then query the database to extract the record for Hungary and display the data in a dataframe. [1 point]

```
[72]: collection.update_one(
    {"country": "Hungary"},
    {"$set": {"democracy": 0.4}}
)

hungary_record = collection.find(
    {"country": "Hungary"},
    {"_id": 0}
)

df_hungary = pd.DataFrame(loads(dumps(hungary_record)))

display(df_hungary)
```

			text	country	adopted	\
0	'Hungary 2011 (rev. 2013)	Preamble	\nGo...	Hungary	2011	
	revised	reinstated	democracy			
0	2013.0	None	0.4			

Part d Set the `text` field in the database as a text index. Then query the database to find all constitutions that contain the exact phrase “freedom of speech”. Display the country name, adoption year, and democracy scores in a dataframe for the constitutions that match this query. [2 points]

```
[73]: collection.create_index([("text", "text")])

query = {
    "$text": {
        "$search": "\"freedom of speech\""
    }
}

projection = {
    "_id": 0,
    "country": 1,
    "adopted": 1,
    "democracy": 1
}

results = collection.find(query, projection)

df_results = pd.DataFrame.loads(dumps(results))

display(df_results)
```

		country	adopted	democracy
0		Slovenia	1991	0.861380
1		Poland	1997	0.682208
2		Eritrea	1997	0.075621
3		Croatia	1991	0.710922
4	Macedonia (The former Yugoslav Republic of)		1991	0.510983
5		Kazakhstan	1995	0.262596
6		Zimbabwe	2013	0.315359
7		Kenya	2010	0.531911
8		Fiji	2013	0.473559
9		Finland	1999	0.856265
10		Georgia	1995	0.757486
11		Namibia	1990	0.745421
12		Spain	1978	0.834466
13		Korea (Republic of)	1948	0.757692
14		Antigua and Barbuda	1981	NaN

15	Saint Vincent and the Grenadines	1979	NaN
16	Dominica	1978	NaN
17	Swaziland	2005	0.136008
18	Ghana	1992	0.670849
19	Gambia	1996	0.348132
20	Peru	1993	0.730816
21	Rwanda	2003	0.274476
22	Papua New Guinea	1975	0.488884
23	Sierra Leone	1991	0.564657
24	Belize	1981	NaN
25	Hungary	2011	0.400000
26	Seychelles	1993	0.589139
27	China	1982	0.096066
28	Saint Kitts and Nevis	1983	NaN
29	Uganda	1995	0.338308
30	Bangladesh	1972	0.369978
31	Somalia	2012	0.177772
32	South Africa	1996	0.727070
33	Trinidad and Tobago	1976	0.730927
34	Jordan	1952	0.270614
35	Samoa	1962	NaN
36	Sri Lanka	1978	0.647035
37	Liberia	1986	0.629972
38	Mexico	1917	0.672567
39	Lao People's Democratic Republic	1991	0.094434
40	Pakistan	1973	0.430273
41	India	1949	0.632527
42	Myanmar	2008	0.405772
43	Bhutan	2008	0.537041
44	Korea (Democratic People's Republic of)	1972	0.090438
45	Philippines	1987	0.567393
46	Tonga	1875	NaN
47	Marshall Islands	1979	NaN
48	Cyprus	1960	0.810509
49	Singapore	1963	0.446464
50	Malaysia	1957	0.345091
51	United States of America	1789	0.849155

Part e Use a query to search for the terms “freedom”, “liberty”, “legal”, “justice”, and “rights”. Generate a text score for all of the countries, and display the data for the countries with the top 10 relevancy scores in a dataframe. [2 points]

```
[74]: collection.create_index([("text", "text")])

query = {
    "$text": {
        "$search": "freedom liberty legal justice rights"
```

```

    }
}

projection = {
    "_id": 0,
    "country": 1,
    "adopted": 1,
    "democracy": 1,
    "score": {"$meta": "textScore"}
}

results = collection.find(query, projection).sort([("score", {"$meta": "textScore"})]).limit(10)

df_top10 = pd.DataFrame.loads(dumps(results))

display(df_top10)

```

	country	adopted	democracy	score
0	Serbia	2006	0.474443	5.030999
1	Finland	1999	0.856265	5.029000
2	Estonia	1992	0.909233	5.024473
3	Armenia	1995	0.393278	5.023651
4	Albania	1998	0.535111	5.023087
5	Dominican Republic	2015	0.583654	5.019910
6	Moldova (Republic of)	1994	0.571357	5.017063
7	El Salvador	1983	0.661989	5.016899
8	Georgia	1995	0.757486	5.015282
9	Turkey	1982	0.341745	5.014672

1.1.9 Question 3

Close the connections to the PostgreSQL and MongoDB databases. [1 point]

```

[79]: engine.dispose()
print("PostgreSQL connection closed.")

client.close()
print("MongoDB connection closed.")

```

PostgreSQL connection closed.
MongoDB connection closed.